Certified Constant-Shape Usage for Bounded LLM Workflows: Source-Bound Validation and Static Admission in Lean

Nambi Rajan M[a], Ranjithkumar S[a]

[a] *Vellore Institute of Technology, Vellore, India*

Abstract

Usage logs of LLM workflows record model identities and token counts even when prompts are masked, and these metadata reveal which path a workflow took. OrchLang is a bounded workflow language whose untrusted compiler pads calls, aligns secret-dependent branches and completes loops so that the logical usage trace does not depend on secrets. A Lean validator checks each repair against an original program and secrecy policy that a trusted reader re-derives from source. A decidable static certificate guarantees before execution that every reached request fits its envelope, so the released trace is the work actually performed and no run that passes the input guard is refused midway. In 2,312 lines of Lean we prove usage noninterference, functional preservation and exact per-model budgets under restricted-domain serving contracts that a non-truncating padder satisfies. For a UTF-8 byte interface the structural envelope obligations are theorems, and the tokenizer bound reduces to a segmentation property that the runtime checks before every dispatch. On twelve workflow variants and three local models, a heuristic study of 1,684 pairs and a certified study of 1,682 pairs preserve every saved output and emit one logical trace per workflow. Checking the Python–Lean boundary exposed and fixed a Unicode serialization defect. Timing, server-side batch metadata and physical failures remain outside the guarantee.

Keywords: workflow languages, information flow, noninterference, translation validation, resource admission, Lean, LLM systems

Email addresses: nambirajan.m2024@vitstudent.ac.in (Nambi Rajan M), ranjithkumar.s@vit.ac.in (Ranjithkumar S)

1. Introduction

LLM applications are increasingly built as workflows: a router classifies a request, a specialist model answers on one branch, an evaluator loops until a draft passes. Operators export usage records of these workflows to observability and billing systems. Tracing conventions record model names and input/output token counts per span [1], and observability products offer masking of prompt and response contents while still recording that usage metadata [2, 3, 4]. Masking contents does not mask control flow. If a medical query takes a branch that calls a larger model twice, the ordered list of model identities and token counts reveals the branch. Token counts and routing metadata are established leakage channels for single requests [5, 6, 7]; a workflow multiplies them across every call it makes.

The programming problem is to change a whole bounded workflow so that its usage record no longer depends on secret inputs, while every decision the workflow makes stays the same. Constant-resource programming, control-flow balancing, padding and translation validation solve analogous problems for timing and network traffic [8, 9, 10, 11]. LLM workflows add three difficulties. First, an operation is a model contract (weights, tokenizer, chat template, stop policy and decoding grammar), and counts in one contract's unit say nothing about another's. Second, later prompts are built from earlier generated text, so whether a padding target is large enough depends on values that do not yet exist when the workflow starts. Third, a validator's guarantee is only as good as the original program and secrecy policy it is given.

OrchLang addresses these difficulties as follows. An untrusted compiler proposes padding targets, dummy calls and loop completion. A trusted source reader re-derives the original program and the declared secrets from the source bytes, and a Lean validator checks the candidate against them (Section 4.1). A decidable static certificate bounds every value from which a request is built, which proves before execution that every padded request reached by any trajectory fits its target (Section 4.3). The runtime admits complete requests, reserves per-model budgets and records request hashes and generated token IDs.

What the guarantee buys. A trusted wrapper could simply print a fixed report. Such report normalization still needs everything OrchLang checks: a static shape to report, evidence that the repair preserves the original's decisions, and evidence that every request fits its slot. Without the last item,

a report lies whenever a request outgrows its slot, or the workflow must refuse midway and the refusal itself leaks. Executing the shape makes the report *truthful*: it equals the logical work performed in each model contract's own unit and reconciles against local evaluation records, which matters for budget enforcement and audit. The guarantee concerns this released logical record. A server that observes physical batches, a clock or packets needs a separate serving refinement, and we measure how far one prototype gets (Section 6.5).

Contributions.

1. *Source-bound translation validation* for LLM workflows. Because the original program and policy come from source rather than from the candidate, a repair cannot weaken the policy it is checked against (Section 4.1).
2. *A restricted-domain mechanized theory.* Noninterference, functional preservation and exact per-model budgets hold under serving contracts that constrain only admitted requests. A concrete non-truncating padder satisfies these contracts and provably violates the global ones used in earlier formulations (Section 4.2).
3. *A static admission certificate* whose soundness is proved in Lean, and whose byte-interface instance reduces all remaining tokenizer obligations to a segmentation property monitored before each dispatch (Sections 4.3 and 4.4). The end-to-end theorems need only an input guard.
4. *Boundary assurance* for the trusted Python implementation: three-way operational agreement with Lean's executable semantics, and an exact serialization round trip that found and fixed a real defect (Section 5).
5. *An evaluation* on twelve workflow variants and three local models. It comprises a heuristic study of 1,684 pairs, a certified study over every guard-admitted input, cost measurements, and a native single-batch prefill prototype whose negative results we report (Section 6).

2. Observer and threat model

Observation. The observer receives the ordered list of events exported for one workflow run,

$$e ::= \mathsf{llm}(m, t_{\text{in}}, t_{\text{out}}) \mid \mathsf{tool}(f),$$

where m is a model contract and f a tool name. Contents, call-site identifiers, dummy flags, natural stopping lengths and private audit records are not

Table 1: Inherited mechanisms and the additional contract studied here.

Prior method	Established contribution	Contract added by OrchLang
Agat, constant-resource types, FaCT [8, 9, 12]	Balance control paths; constrain resource-dependent observations.	Balance across separately identified model contracts with visible caps and generated-text dependencies.
Translation validation [11, 13]	Check each translation against a supplied original.	The original and secrecy policy are re-derived from source bytes, not supplied by the candidate.
Supersequence defenses, Pacer [10, 14]	Shape traffic under a network observer.	Ordered model/tool events with exact logical counts; useful-call order is preserved.
RelCost, eRHL [15, 16]	General relational cost reasoning.	A decidable admission certificate with a Lean soundness proof, specialised to text-valued stores.
Token-budget tools [17, 18]	Bound or catalogue agent spending.	Per-model budgets that are exact consequences of a certified shape.

exported. We assume the trusted wrapper releases the record once per run, or with timestamps normalised to the run, so that per-event latency is not part of the observation. This is an explicit deployment requirement. A retained latency-augmented attack reaches 100 percent balanced accuracy against one shaped workflow (Section 6.4), so constant logical usage does not imply constant running time.

Property. For stores $\sigma \equiv_{\mathsf{L}} \tau$ that agree on public variables, *usage noninterference* requires equal traces for every fixed randomness tape. Public inputs may still select different traces. Equality for every tape implies equality of trace distributions under any secret-independent tape distribution.

Trust. The model and the trusted host may see secret contents. The source reader, lowerer, serializer, runtime and native serving code are trusted. The compiler is untrusted. Figure 1 summarises which interfaces carry which evidence; Section 5 lists the trusted computing base.

Out of scope. Elapsed time, streaming chunk boundaries, packets, shared prompt or KV caches, concurrency between runs, and physical failures are

Interface	Assurance
Checked core	Lean theorems: restricted-domain noninterference, preservation, budgets, static admission (Section 4).
Byte-bounded adapter	UTF-8 envelope lemmas proved; tokenizer segmentation and input guard checked at run time.
Python implementation	Three-way operational agreement; exact serialization round trip; tests.
Native serving	Logical accounting reconciled against eval/yield records; no physical-observer refinement.

Figure 1: Assurance by interface. A check at one interface does not certify the interfaces below it.

outside the observation. So are external tool providers: a dummy tool event is a logical record and does not contact a real service. In isolated-padding mode the wrapper reports filler and request prefill as one input count, but an observer below the wrapper sees two submissions and recovers the true request length. Section 6.5 measures a single-submission prototype for that lower observer.

Admission and refusal. Before dispatch the host counts the fully rendered request in the receiving model's unit and refuses overflow rather than truncating. A refusal is itself observable. Our certified guarantees therefore include admission: under the static certificate, no request reached by any guarded trajectory is refused (Section 4.3). The input guard is a pre-dispatch decision on input sizes against public byte bounds. Inputs it rejects are refused before any model call, so a refusal reveals only whether some input exceeds its declared bound. That one bit is outside the theorems and is the deployment's responsibility, for example by rejecting oversized inputs at the user interface.

3. Language and shaping

3.1. Bounded programs

The source language has labelled text inputs, model profiles, templated prompts, capped calls, pure tools, conditionals and bounded loops. For example:

```
workflow Router {
  input q: text secret max_tokens 48;
  model m = "llama1b";
  prompt classify(x) = user "Classify: {x}";
  prompt respond(x) = user "Answer: {x}";
  let route = call classify(q) using m max_tokens 3;
  if route == "short" {
    let answer = call respond(q) using m max_tokens 16;
  } else {
    let answer = call respond(q) using m max_tokens 64;
  }
  output answer;
}
```

Unshaped usage exposes the request lengths and the selected cap. Shaping fixes the classification usage and aligns the two response arms while keeping their visible caps. The core contains sequencing, calls, tools, dummies, conditionals and loops. Expressions contain literals, variables, concatenation, tool-argument escaping and template-level trimming. Escaping prevents separator characters or backslashes from changing argument boundaries. Lean defines it character-wise; an executable check confirms agreement with the adapter's sequential replacement.

Labels form $\mathsf{L} \sqsubseteq \mathsf{H}$. Flow-insensitive propagation joins argument and program-counter labels. A high call requires fixed usage, a high conditional requires equal arm shapes, and a high loop requires a shaped body completed to its public bound. Model responses inherit the label of their requests. Figure 2 gives the complete core grammar, using the constructors of the executable Lean definitions.

3.2. Targets, padding and output

In heuristic mode the compiler sizes targets from tokenized literals, variable caps and slack. This is an execution envelope, not a sound bound: subword concatenation and cross-model values invalidate scalar token bounds. In one audit example, a string counted as one BERT token needed 259 GPT-2 tokens in its complete request, against an 18-token target. Exact complete-request admission catches such cases at run time. Certified mode (Section 4.3) replaces the heuristic with sizes that are proved sufficient.

Isolated padding prefills filler in a separate context, resets it and evaluates the unchanged request. Logical input usage is the sum of filler and request

$$
\begin{array}{rcl}
\ell & ::= & \mathsf{L} \mid \mathsf{H} \qquad (\mathsf{L} \sqsubseteq \mathsf{H}) \\
e & ::= & s \mid x \mid e \mathbin{+\!\!+} e \mid \mathsf{escape}(e) \mid \mathsf{trim}(e) \\
g & ::= & x = s \mid \mathsf{prefix}(x, s) \\
\pi & ::= & \bot \mid (T, N) \\
p & ::= & \mathsf{skip} \mid p; p \\
 & & \mid \mathsf{call}_i(x, m, k, e, \pi) \mid \mathsf{tool}(x, f, e) \\
 & & \mid \mathsf{dummy}(m, T, N) \mid \mathsf{dummyTool}(f) \\
 & & \mid \mathsf{if}\ g\ \mathsf{then}\ p\ \mathsf{else}\ p \\
 & & \mid \mathsf{loop}(n, g, p, b), \qquad b \in \{\mathsf{false}, \mathsf{true}\}.
\end{array}
$$

Strings are s; variables and randomness sites are natural numbers x, i. The model contract is m, the useful output cap is k, and padding annotation (T, N) fixes logical input and output counts. The loop runs at most n times, stops when g holds, and emits cover iterations when $b = \mathsf{true}$. This grammar lists every constructor of Lean's `Expr`, `Guard` and `Stmt`; $\pi = \bot$ denotes an unpadded call.
The corresponding surface fragments are

```
input x: text secret max_tokens c;
prompt P(x) = user "template {x}";
let y = call P(x) using m max_tokens k;
let y = tool f(x);
if x == "value" { ... } else { ... }
repeat n until x == "done" { ... }
```

The trusted frontend expands templates into expressions, allocates slots and sites, and joins labels into a fixed context Γ. Literal model/template framing is part of the request expression rather than an implicit token charge.

Figure 2: Complete core grammar and representative surface constructs. Surface parsing and lowering are tested implementations outside the Lean proof.

counts. In-band filler changes the visible prompt and may change responses, so it is not the default. Forced decoding generates exactly N token IDs, continuing past a stop token. The stop token is not masked: generation simply proceeds from the context that contains it, so tokens after the first stop never enter the visible response. The workflow receives the prefix before the first stop, cut to its visible cap $k \leq N$. Natural calls exclude the terminating stop ID; forced calls count all N IDs. The implementation recognises EOS and declared single-token stop IDs and rejects multi-token stop strings before execution, because string-level matching across a visible-cap boundary is outside the semantics.

3.3. Alignment and loops

A two-sequence dynamic program aligns compatible model contracts or equal tool names, merges counts by componentwise maximum and inserts dummy events for missing positions. With weight $t_{\text{in}} + 4t_{\text{out}}$ per contract it minimises the common two-arm profile cost. The weight is a chosen, decode-sensitive objective rather than a measured price; Section 6.4 varies it. Multi-arm alignment is progressive, and global optimality is not claimed. After early loop termination, ghost iterations emit the body's shape without updating the store or consuming real-site randomness. Model dummies perform real local generation; dummy tool events only add records. Figure 3 gives the operational semantics.

4. Mechanized guarantees

4.1. Acceptance from a bound source

Translation validation checks each proposed transformation rather than the optimiser [11, 13]. Lean checks four conditions on a candidate p for original p_0 under labelling Γ:

- the padding discipline;
- separated high and public randomness sites;
- visible caps no larger than forced lengths;
- equality of normalised erasures, so useful requests cannot be changed, reordered or dropped.

Write $\langle p, \sigma, c\rangle \Downarrow_{w,\omega} \langle \sigma', c', t\rangle$ for the store, site counters and emitted trace. Let $r = [\![e]\!]_\sigma$ and $a = \omega(i, c(i))$. The rules below abbreviate the recursive function `exec`.

$$\frac{y = \mathsf{gen}_m(r, k, a)}{\langle \mathsf{call}_i(x, m, k, e, \bot), \sigma, c\rangle \Downarrow \langle \sigma[x \mapsto y], c[i \mapsto c(i)+1], [\mathsf{llm}(m, \mathsf{tok}_m(r), \mathsf{outTok}_m(y))]\rangle}$$

$$\frac{r' = \mathsf{padReq}_m(T, r) \quad y = \mathsf{forced}_m(r', a, N)}{\langle \mathsf{call}_i(x, m, k, e, (T, N)), \sigma, c\rangle \Downarrow \langle \sigma[x \mapsto \mathsf{view}_m(k, y)], c[i \mapsto c(i)+1], [\mathsf{llm}(m, \mathsf{tok}_m(r'), \mathsf{outTok}_m(y))]\rangle}$$

$$\frac{\langle p, \sigma, c\rangle \Downarrow \langle \sigma_1, c_1, t_1\rangle \quad \langle q, \sigma_1, c_1\rangle \Downarrow \langle \sigma_2, c_2, t_2\rangle}{\langle p; q, \sigma, c\rangle \Downarrow \langle \sigma_2, c_2, t_1 +\!\!+ t_2\rangle}$$

$$\frac{[\![g]\!]_\sigma = \mathsf{true} \quad \langle p, \sigma, c\rangle \Downarrow \langle \sigma', c', t\rangle}{\langle \mathsf{if}\ g\ \mathsf{then}\ p\ \mathsf{else}\ q, \sigma, c\rangle \Downarrow \langle \sigma', c', t\rangle}$$

The false-guard conditional rule selects q. A tool writes $\mathsf{toolFn}_f(r)$ to x, leaves counters unchanged and emits $[\mathsf{tool}(f)]$. A model dummy emits its padded/forced event at cell 0 without updating the store or counters; a tool dummy emits only $[\mathsf{tool}(f)]$.

$$\frac{\neg[\![g]\!]_\sigma \quad \langle p, \sigma, c\rangle \Downarrow \langle \sigma_1, c_1, t_1\rangle \quad \langle \mathsf{loop}(n, g, p, b), \sigma_1, c_1\rangle \Downarrow \langle \sigma_2, c_2, t_2\rangle}{\langle \mathsf{loop}(n+1, g, p, b), \sigma, c\rangle \Downarrow \langle \sigma_2, c_2, t_1 +\!\!+ t_2\rangle}$$

A zero-bound loop emits []. If its guard already holds, an unpadded loop emits []; a padded loop emits n copies of the body's ghost trace and leaves the store and counters unchanged. Under conformance on the cover domain, that ghost trace equals $\mathsf{shape}(p)$. These rules include no physical exceptions.

Figure 3: Operational semantics, with the fixed world and tape omitted from rule arrows for readability. An initial accumulated trace is prepended to t.

Figure 4 gives the rules. The question a validator does not answer is where p_0 and Γ come from. OrchLang's trusted source reader derives the original, slots, labels and declared secret set from the source bytes, independently of the candidate. The interface requires every declared secret to be labelled high.

This binding matters in practice. For a source-declared secret query, a candidate that supplies an empty secret list and an unshaped call is *accepted* by the core checker under the weakened all-public policy. The source-bound entry point re-derives the declaration, rejects the same call, and accepts the compiler's padded repair. The executable `experiments/-source_binding_case.py` reproduces all three Lean verdicts. Only SHA-pinned model aliases with identical template, stop, grammar and padding contracts share a model identity.

4.2. Worlds and restricted serving contracts

Execution is parameterised by a world w providing tokenization tok_m, natural and forced generation, padding padReq_m, visible-prefix extraction view_m and pure tools. A tape $\omega(i, j)$ supplies randomness to the jth execution of call site i; dummies and ghost iterations do not advance site counters. A *request domain* $D(m, T, N, r)$ names the padded calls a serving adapter supports: model m, input target T, forced length N, unpadded request r.

Definition 1 (Restricted serving contracts)**.** $\mathsf{ConformsOn}(w, D)$ holds if for all $(m, T, N, r) \in D$ and every cell c:

$$\mathsf{tok}_m(\mathsf{padReq}_m(T, r)) = T, \qquad \mathsf{outTok}_m(\mathsf{forced}_m(\mathsf{padReq}_m(T, r), c, N)) = N.$$

$\mathsf{TransparentOn}(w, D)$ holds if for all $(m, T, N, r) \in D$, all $k \le N$ and all c:

$$\mathsf{view}_m(k, \mathsf{forced}_m(\mathsf{padReq}_m(T, r), c, N)) = \mathsf{gen}_m(r, k, c).$$

The trajectory predicate $\mathsf{RunsIn}(w, \omega, D, p, s)$ holds when every padded call and dummy actually reached by executing p from state s lies in D, including loop-completion calls. It follows evaluated intermediate stores and guards.

Earlier formulations, including our own, assumed the global contracts $\mathsf{Conforms}(w)$ and $\mathsf{Transparent}(w)$, which quantify over every T, including targets smaller than the request. The following proposition shows why the restriction matters.

Let $L_\Gamma(e)$ join variable labels, and let $\ell = pc \sqcup L_\Gamma(e)$ and $pc' = pc \sqcup \Gamma(\mathsf{guardVar}(g))$. The judgment $\Gamma; pc \vdash p$ abbreviates `wt` returning true.

$$\frac{\ell \sqsubseteq \Gamma(x) \quad (\ell = \mathsf{H} \Rightarrow \pi \neq \bot)}{\Gamma; pc \vdash \mathsf{call}_i(x, m, k, e, \pi)} \qquad \frac{pc \sqcup L_\Gamma(e) \sqsubseteq \Gamma(x)}{\Gamma; pc \vdash \mathsf{tool}(x, f, e)}$$

$$\frac{\Gamma; pc' \vdash p \quad \Gamma; pc' \vdash q \quad pc' = \mathsf{H} \Rightarrow \mathsf{shape}(p) = \mathsf{shape}(q) \neq \bot}{\Gamma; pc \vdash \mathsf{if}\ g\ \mathsf{then}\ p\ \mathsf{else}\ q}$$

$$\frac{\Gamma; pc' \vdash p \quad pc' = \mathsf{H} \Rightarrow (b = \mathsf{true} \wedge \mathsf{shape}(p) \neq \bot)}{\Gamma; pc \vdash \mathsf{loop}(n, g, p, b)}$$

Skip and dummies are well typed; sequencing requires both components to be well typed. Separate structural checks require $k \leq N$ at padded calls and mark sites reached under a high program counter. Let H be that site set.

$$\frac{\Gamma; \mathsf{L} \vdash p \quad \mathsf{sitesOK}(\Gamma, H, \mathsf{L}, p) \quad \mathsf{capsOK}(p) \quad \mathsf{clean}(\lfloor p \rfloor) = \mathsf{clean}(\lfloor p_0 \rfloor)}{\mathsf{validateProgram}(\Gamma, p_0, p) = \mathsf{true}}$$

Erasure removes padded annotations and dummies, preserves useful calls/tools and control flow, and clears loop-completion flags. Normalization `clean` removes skip/sequence artifacts without changing useful order.

$$\frac{\mathsf{readSource}(b) = (p_0, \Gamma, S) \quad \forall x \in S,\ \Gamma(x) = \mathsf{H} \quad \mathsf{validateProgram}(\Gamma, p_0, p) = \mathsf{true}}{\mathsf{acceptSource}(b, p)}$$

The last rule describes the trusted Python entry point. Its source reader, unlike the candidate, supplies the original and the declared secret set S. Lean checks the core premises, but does not prove that `readSource` implements parsing.

Figure 4: Label propagation, padding discipline, erasure and source-bound acceptance. The displayed rules match the executable checks rather than adding a second type system.

Proposition 1 (Non-vacuity)**.** *Let* padWorld *count characters, pad by appending filler without truncating, and force exactly N output tokens. For every envelope b it satisfies* ConformsOn(padWorld, D_b) *for the envelope domain D_b defined below and* TransparentOn(padWorld, D) *for every D. It does not satisfy the global* Conforms*.*

The global premise therefore excludes the very padders we deploy, while the restricted premises admit them. All theorems below use only the restricted contracts. The global versions remain available as corollaries (`World.Conforms.on`, `World.Transparent.on`).

Theorem 1 (Restricted-domain noninterference)**.** *Let p pass the padding discipline and randomness-site check, and let* ConformsOn(w, D)*. For public-equal states a, b with* RunsIn(w, ω, D, p, a) *and* RunsIn(w, ω, D, p, b)*, the two executions end in public-equal states with equal traces.*

The proof inducts over statements with the relation "public-equal stores, counters equal off high sites, equal traces". Public calls use equal requests and cells. High calls emit (T, N) because their requests are admitted. High branches and loops emit their common shape by the restricted shape lemma, and confinement preserves public state under a high program counter. Unlike a constant-shape argument, Theorem 1 covers programs whose trace depends on public inputs.

4.3. Static admission certificate

RunsIn is a trajectory property: a later prompt depends on generated text, so checking it would require predicting model outputs. The certificate avoids lookahead by bounding the values from which every request is built. Let $|r|_b$ be a text measure and let the finite vector v bound every store slot, so that the store invariant is $I_v(\sigma) \equiv \forall x,\ |\sigma(x)|_b \le v_x$. The computable expression envelope is

$$\begin{aligned} E_v(s) &= |s|_b, & E_v(x) &= v_x, \\ E_v(\mathsf{escape}(e)) &= 2E_v(e), & E_v(\mathsf{trim}(e)) &= E_v(e), \\ E_v(e_1{+}{+}e_2) &= E_v(e_1) + E_v(e_2). \end{aligned}$$

An *envelope* b supplies the measure together with model constants and bounds. Its soundness Sound(b, w) has six conjuncts:

- three structural ones: subadditive concatenation, at most twofold escaping and nonexpansive trimming;
- tokenizer bounds $\mathsf{tok}_m(r) \le \alpha_m |r|_b + \delta_m$;
- visible-response bounds $\rho_m(k)$;
- tool-result bounds θ_f.

The envelope domain is $D_b(m, T, N, r) \equiv \mathsf{tok}_m(r) \le T \wedge T + N \le C_m$. For each padded call $\mathsf{call}_i(x, m, k, e, (T, N))$ the checker `staticCertificate` tests

$$\alpha_m E_v(e) + \delta_m \le T, \qquad T + N \le C_m, \qquad k \le N, \qquad \rho_m(k) \le v_x,$$

requires $\theta_f \le v_x$ for tool assignments, checks dummies against their fixed request, and recurses into both branches and loop bodies. Padded loops also check their cover profile. The checker never calls `exec` or RunsIn.

Theorem 2 (Static certificate implies admission)**.** *If* $\mathsf{Sound}(b, w)$ *and* `staticCertificate` *accepts* p *under* v*, then for every store* σ *with* $I_v(\sigma)$ *and every tape* ω*,* $\mathsf{RunsIn}(w, \omega, D_b, p, \sigma)$ *holds, and the final store satisfies* I_v*.*

The proof carries I_v through every useful assignment, using the response and tool bounds, and leaves it unchanged under cover work. Loop cases use an invariant lemma for both real and ghost iterations.

Certified end-to-end guarantees. Composing Theorem 2 with the restricted theory removes every trajectory premise. In the following, p is accepted by `validateProgram` against the source-derived original p_0 and `staticCertificate` accepts p under v.

Theorem 3 (Certified noninterference)**.** *If* $\mathsf{Sound}(b, w)$ *and* $\mathsf{ConformsOn}(w, D_b)$*, then for all public-equal* σ, τ *satisfying* I_v *and every tape* ω*, executing* p *from* σ *and from* τ *emits the same trace. In particular, stores that differ only in declared secret inputs emit the same trace.*

Theorem 4 (Certified preservation)**.** *If* $\mathsf{Sound}(b, w)$ *and* $\mathsf{TransparentOn}(w, D_b)$*, then for every* σ *satisfying* I_v *and every* ω*,* p *and the erasure of* p_0 *end in the same store.*

Theorem 5 (Certified shape and budgets)**.** *If* $\mathsf{Sound}(b, w)$*,* $\mathsf{ConformsOn}(w, D_b)$ *and* $\mathsf{shape}(p) = s$*, then every guarded execution emits exactly* s*. Let* $U_m(s)$ *be the input-token sum, output-token sum and call count of contract* m *in* s*. If* $U_m(s) \leq B_m$ *componentwise for every* m*, every guarded execution stays within the budgets* B*.*

The only run-time premise left is the input guard $I_v(\sigma)$, which is decided on input sizes before the first dispatch. Budgets are kept per contract because token units of different models are incomparable. Accounting alone does not replace the ordered shape: paths $[A, B]$ and $[B, A]$ have equal per-contract totals but different traces.

4.4. Discharging the byte envelope

The deployed adapter measures text in UTF-8 bytes. Its structural obligations are proved rather than assumed.

Lemma 1 (UTF-8 envelope)**.** *For* $|\cdot|_b = \mathsf{utf8ByteSize}$*: concatenation is additive, escaping at most doubles size, and trimming does not increase it. Hence* $\mathsf{Sound}(b, w)$ *reduces to the tokenizer, response and tool bounds.*

The response and tool bounds are enforced by the adapter's post-decoding byte buffers, which both the original and the shaped program use. The response buffer allows $128k$ bytes for visible cap k, the largest raw token-piece length in all three pinned vocabularies (Section 6.3). The remaining assumption, $\alpha_m = 1$ and $\delta_m = 0$, reduces to a property of a single tokenization.

Lemma 2 (Segmentation bound)**.** *If a tokenizer emits pieces that are each nonempty and whose concatenation equals the input bytes, then the number of tokens is at most the number of input bytes.*

Byte-level BPE with automatic BOS insertion disabled has this form. The chat framing, including the begin-of-text marker, is part of the request expression, and special tokens render as their own nonempty literals. The certified runtime checks both premises on every complete request before dispatch: it reads each native piece with the special-token convention, compares their concatenation with the request bytes, and refuses a mismatch without dispatching. The tokenizer bound is therefore monitored rather than assumed. What remains trusted is that the native piece table describes the token IDs the model consumes.

4.5. Refusal and ideal admission

Without a certificate, the per-call admission check prevents overspending but leaves a prefix channel. A first call can complete before a later rendered request exceeds its target, so one run is refused after $[e_1]$ while another completes $[e_1, e_2]$. An executable falsification exhibits exactly this with two public-equal stores. We therefore keep an ideal admission semantics with observations $\mathsf{refused}$, $\mathsf{completed}(t)$, $\mathsf{failed}(\mathit{prefix})$ and $\mathsf{unknownDispatch}(\mathit{prefix})$.

Proposition 2. *If* $\mathsf{shape}(p) = s$ *and* $\mathsf{ConformsOn}(w, D)$*, every execution satisfying* RunsIn *emits exactly* s*. If* RunsIn *holds for every advertised input and tape, every ideal observation is* $\mathsf{completed}(s)$*.*

The first part is the restricted shape lemma used throughout. The second is discharged for guarded inputs by Theorem 2. Transport failures and unknown dispatches are outside the theorems; the runtime retains reservations when dispatch usage cannot be confirmed.

4.6. Mechanization

Table 2 maps every result to its Lean declaration. The development has 2,312 lines and 78 theorems in seven files and is checked by Lean 4.19.0 [19] with no `sorry`. The script `publication/check_proofs.py` compiles the files in dependency order and fails if any declaration depends on an axiom other than `propext`, `Quot.sound` or `Classical.choice`. Only the ideal-admission observations use `Classical.choice`, because RunsIn is not decidable. All certified theorems use only `propext` and `Quot.sound`.

5. Implementation and trusted computing base

The implementation is a Python frontend, compiler and runtime of about 2,600 lines over llama-cpp-python, plus the Lean development. The runtime admits complete requests, reserves per-model usage before dispatch and confirms counts and caps on return. A transport exception or a nonconforming return leaves the reservation in place, so usage is marked incomplete rather than recorded as a zero-cost success. Each run manifest binds source, dataset, selected IDs, implementation hashes, model contracts, decoding parameters, platform and native-library hashes. Resuming requires an identical manifest. A separate `python -m orchlang check` command re-reads source and model bytes to check an exported artifact without trusting its declared original or secrets.

Table 2: Results and their Lean declarations (file prefix `proof/`).

Result	Declaration	File
Prop. 1	`padWorld_conformsOn`, `padWorld_transparentOn`, `padWorld_not_globally_conforming`	RestrictedDomain
Thm. 1	`noninterference_on`, `trace_noninterference_on`	RestrictedDomain
Thm. 2	`static_certificate_sound`, `static_certificate_implies_RunsIn`	AdmissionDomain
Thm. 3	`certified_noninterference`, `certified_secret_inputs_sound`	RestrictedDomain
Thm. 4	`certified_preservation` (via `erase_correct_on`)	RestrictedDomain
Thm. 5	`certified_shape_exact`, `certified_budget_sound`	RestrictedDomain
Lemma 1	`utf8_append`, `utf8_escape`, `utf8_trim`, `utf8_envelope_sound`	Utf8Envelope
Lemma 2	`segmentation_count_le`, `tokenizer_alpha_one`	Vocabulary-Envelope
Prop. 2	`shape_exec_on`, `certified_domain_noninterference`	AdmissionDomain
Global corollaries	`noninterference`, `erase_correct`, `shape_budget_sound`, `validateProgram_sound`	ShapeCore

5.1. Checking the Python–Lean boundary

Lean checks the program it parses, so the serializer is part of the trusted base. We added an encoder to the Lean development and a `roundtrip` entry point that parses each serialized program and re-encodes it. Programs come from the differential generator, with literals extended by accented, CJK, emoji, backslash, unit-separator, quote, line-separator and special-token strings, and from every application source with its heuristic and certified candidates.

The first run matched only 2,517 of 3,036 programs. Python's JSON encoder escaped characters outside the Basic Multilingual Plane as UTF-16 surrogate pairs, and Lean's parser decoded each surrogate separately into U+0000. Lean was therefore checking a different literal. Besides the validator, this affected the static certificate, which would have under-counted a four-byte character as two bytes. No shipped workflow source contains a non-ASCII character, so no reported result was affected. After switching both interfaces to raw UTF-8, all 3,036 programs round-trip exactly. The pre-fix behaviour is reproducible with `-ascii-escaped`, and a regression test covers it. The

Table 3: Trusted computing base and how each part is checked.

Component	Evidence
Source reader, lowerer	Re-derives original and secrets; policy-omission case; tests.
Python → Lean serializer	Exact round trip of 3,036 programs (Section 5.1).
Python runtime semantics	Agreement with Lean `exec` and an independent interpreter on 6,320 executions.
Byte adapter guards	Input guard and post-decoding buffers; refusal-before-dispatch tests.
Native tokenizer	Segmentation premise of Lemma 2 checked per request; exhaustive vocabulary scan.
Native generation	Forced-length and visible-prefix contracts checked per call; eval/yield reconciliation.

episode is a concrete case for treating the serializer as part of the trusted base and checking it mechanically.

6. Evaluation

We answer five questions.

RQ1 Does the trusted implementation agree with the mechanized semantics?

RQ2 Does heuristic shaping preserve outputs and remove trace leakage on real models at scale?

RQ3 Do the certified guarantees hold in execution over every guard-admitted input?

RQ4 What does shaping cost, in heuristic and certified mode?

RQ5 How far does the logical guarantee extend toward the serving layer?

All inference is local and greedy, and every attempted pair stays in its denominator.

Setup. The applications are bounded ports of nine public workflow patterns: routing, prompt chaining, evaluator–optimizer, orchestrator–worker, a prompt-flow switch, an email agent, a calculator agent, corrective RAG and adaptive RAG. They run on Llama 3.2 1B Instruct, with Llama 3.2 3B on the adaptive workflow's large-model path [20]. A Qwen 2.5 1.5B replication [21] re-targets

the switch, email and corrective-RAG workflows, for twelve variants in total. All models use Q4_K_M GGUF weights pinned by SHA-256. We used Python 3.12.14, llama-cpp-python 0.3.36 and native llama.cpp build 11490 (`9c2e0e491`), with four CPU threads, batch size 128 and Vulkan offload to an NVIDIA RTX 5050 Laptop GPU. Seeds are fixed by dataset index and call site. The heuristic study ran from a clean checkout of the inference freeze `41a7d81` at a 4,096-token context. The certified study ran from the later clean commit `e90c7bc`, which adds the segmentation monitor, at a 21,504-token context. The analysis script `publication/analyze_round3.py` regenerates every number below from the raw records.

6.1. RQ1: Implementation agreement

Operational correspondence. The Python runtime, an independent interpreter that imports neither compiler nor runtime, and Lean's executable `exec` run 150 generated programs plus a targeted template program in four store contexts, under two finite kernels and five program variants. They agree on all 6,320 executions and 604 validator checks. These audit worlds count characters; they test agreement of the implementations, not the native tokenizer.

Differential testing. On 1,000 generated programs with random secret labellings, Lean accepts all 4,000 shaped candidates. Python's mirror of the discipline agrees with Lean on all 1,000 originals. All 16,000 paired executions have equal traces for public-equal stores and preserve the original store.

Fault injection. Twelve operators model plausible shaping-compiler bugs, for example dropping a padding annotation, shrinking a target or removing a dummy. Applied to the twelve validated programs, they yield 210 mutants. Each mutant is checked by Lean and, independently, executed by a mock-runtime oracle under varied secrets. The oracle finds a leak in 79 mutants, and Lean rejects all 79. In total Lean rejects 192 mutants, and no accepted mutant changes the original's outputs. The runtime refuses to execute the 26 mutants whose forced length undercuts the visible cap, and Lean rejects all 26 by its cap check.

Serialization and tests. The serialization round trip covers 3,036 programs (Section 5.1). All 52 configured tests pass with the pinned models and Lean configured. They include source binding, immutable resumption, model-indexed reservations, unsupported stop declarations, complete token-piece

Table 4: Heuristic study (one frozen implementation). K: observed decision classes. BA: original-trace attack balanced accuracy in percent with a 95% bootstrap interval over cross-validated predictions; p: label-permutation test (200 permutations). Tr.: distinct shaped traces. Cost: shaped/original weighted logical usage. Task: inherited task accuracy with a Wilson 95% interval. (Q): Qwen replication. A dash marks no task scorer or a single decision class.

Workflow	Pairs	K	Orig. BA [95% CI]	p	Tr.	Cost	Task [95% CI]
Routing	150	2	95.4 [91.3, 98.5]	<0.005	1	1.84	65.3 [57.4, 72.5]
Chaining	150	2	100.0 [100.0, 100.0]	<0.005	1	6.10	–
Evaluator	100	3	100.0 [100.0, 100.0]	<0.005	1	2.01	–
Orchestrator	100	3	100.0 [100.0, 100.0]	<0.005	1	1.13	–
PF switch	210	4	26.5 [21.7, 42.2]	0.20	1	1.54	37.6 [31.3, 44.3]
Email	152	4	59.7 [52.0, 67.4]	<0.005	1	1.66	–
Corrective RAG	150	2	75.0 [50.0, 100.0]	<0.005	1	1.75	34.0 [26.9, 41.9]
Calculator	100	3	100.0 [100.0, 100.0]	<0.005	1	13.99	2.0 [0.6, 7.0]
Adaptive RAG	60	6	83.3 [83.3, 100.0]	<0.005	1	2.29	53.3 [40.9, 65.4]
PF switch (Q)	210	4	55.8 [42.6, 69.1]	<0.005	1	5.49	58.6 [51.8, 65.0]
Email (Q)	152	4	74.0 [67.1, 81.1]	<0.005	1	2.13	–
Corrective RAG (Q)	150	1	–	–	1	1.81	38.0 [30.6, 46.0]

decoding, the pre-dispatch byte guard, the segmentation monitor and the astral-character boundary.

6.2. RQ2: Heuristic shaping at scale

The heuristic study contains all 1,172 main and 512 replication pairs at one frozen implementation. All 1,684 pairs complete with equal saved fields, and every shaped trace equals its static shape. An independent audit re-tokenizes all 3,368 complete requests, reconstructs visible prefixes, and reconciles 10,698 useful and cover model calls against per-model ledgers and local eval/yield records. A deliberately corrupted count is detected.

Leakage. A class-balanced random forest predicts each run's decision (its useful-call path and decision variables) from the ordered model/tool identities, counts, event count and call count, with five-fold cross-validation. Table 4 reports its balanced accuracy on original traces. The attack predicts routing, chaining, evaluator, orchestrator and calculator decisions with at least 95 percent balanced accuracy, and adaptive RAG with 83 percent. Corrective RAG, email and the Qwen switch and email variants leak substantially (56 to 75 percent; all $p < 0.005$). The main switch attack is not distinguishable from

chance ($p = 0.20$), and the primary Qwen corrective-RAG port has a single observed decision. These two workflows are worst-case insurance rather than demonstrated leaks. After shaping, every workflow has exactly one trace, so no classifier can do better than a constant prediction. The classifiers illustrate the threat; the theorems carry the guarantee.

Preservation. Shaping preserves task errors as well as successes. The inherited scorers give 65.3, 37.6, 2.0, 34.0 and 53.3 percent for main routing, switch, calculator, corrective RAG and adaptive RAG. These bounded ports with 1–3B models do not reproduce upstream application accuracy, and the calculator remains a control-flow stress test.

6.3. RQ3: Certified execution over every admitted input

Vocabulary scan. For the byte-bounded adapter we first scanned every vocabulary entry of the three pinned GGUFs through the complete native token-piece API: 128,256 entries for each Llama model and 151,936 for Qwen. All three vocabularies have a maximum raw piece length of 128 bytes, and each contains all 256 single-byte pieces. Each Llama vocabulary has 231 pieces longer than 32 bytes, and Qwen has 240. The scan found a real defect: the Python wrapper's fixed 32-byte buffer returned an empty piece for a 128-byte witness. The decoder now retries with the required buffer size. The response buffer is therefore $128k$ bytes for visible cap k. Input and tool buffers are four bytes per declared token annotation, a chosen policy rather than a conversion theorem.

Protocol. For each variant, the frontend derives slot bounds from all source assignments, sizes every target with E_v, exports the candidate and bounds to Lean's `staticCertificate`, and separately runs source-bound validation. All twelve candidates pass both checks at context 21,504. We then ran *every* planned input: the guard refuses oversized inputs before any dispatch, and both arms run under the same byte interface for every admitted input.

Results. Table 5 reports the outcome. Of 1,684 planned inputs, the guard refuses 2 before dispatch, and all 1,682 admitted pairs complete. All 1,682 have equal saved fields, and 1,682 emit their certified shape. Every one of the 10,675 executed model requests passed the segmentation check (10,675 of 10,675), so the tokenizer premise of the certified theorems was never violated. No request was refused after the first dispatch. Response buffers clipped 0 responses and tool buffers clipped 1,920 tool results, affecting 471 pairs. Both

Table 5: Certified study over all planned inputs. Pairs: executed/planned (the remainder is refused by the input guard). Saved, Shape: pairs with equal saved fields and with the certified shape. Seg.: requests whose tokenization passed the segmentation check (Lemma 2). Clip: pairs with any byte clip. Heur.: bounded originals whose saved fields equal the heuristic study's original for the same ID and seed. Cost: certified/original weighted logical usage; /H: certified relative to heuristic shaped usage.

Variant	Pairs	Saved	Shape	Seg.	Clip	Heur.	Max $T+N$	Cost	/H
Routing	150/150	150/150	150/150	600/600	0	150/150	615	3.20	1.74
Chaining	150/150	150/150	150/150	640/640	1	149/150	16,782	104.20	17.08
Evaluator	100/100	100/100	100/100	1,002/1,002	84	44/100	12,771	29.51	14.47
Orchestrator	100/100	100/100	100/100	988/988	83	24/100	727	1.90	1.68
PF switch	210/210	210/210	210/210	420/420	1	209/210	1,081	3.17	2.05
Email	152/152	152/152	152/152	604/604	0	152/152	2,234	3.37	2.03
Corrective RAG	150/150	150/150	150/150	1,500/1,500	116	118/150	7,293	6.47	3.59
Calculator	98/100	98/98	98/98	709/709	1	98/98	2,861	46.77	3.34
Adaptive RAG	60/60	60/60	60/60	1,705/1,705	60	0/60	20,606	40.50	16.43
PF switch (Q)	210/210	210/210	210/210	420/420	9	201/210	928	10.47	1.91
Email (Q)	152/152	152/152	152/152	587/587	0	152/152	2,149	4.20	1.98
Corrective RAG (Q)	150/150	150/150	150/150	1,500/1,500	116	132/150	7,208	6.53	3.53

arms are clipped identically, so preservation is unaffected, but the bounded interface is a different application. Only 1,429 of 1,682 bounded originals have the same saved fields as the unbounded heuristic-study originals with the same seed. The differences concentrate in the workflows whose tools return retrieved passages or long intermediate results (evaluator, orchestrator, corrective and adaptive RAG). There the declared tool annotation, at four bytes per token, is shorter than typical tool output. Adaptive RAG clips a retrieval result in every pair, and none of its 60 bounded originals matches the unbounded one. Routing, switch, email and calculator are essentially unchanged. A tool bound is therefore an application decision: raising it reduces clipping but enlarges every downstream envelope (RQ4). A retained earlier attempt at a 32,768-token context completed 16 pairs before device-memory allocation failed in the adaptive workflow. Token-context admission does not bound device memory.

6.4. RQ4: Cost

Heuristic mode. With input weight one and output weight four, the main-study median cost is 1.84 times original usage, ranging from 1.13 to 13.99. Without the calculator stress test the median is 1.80. Recompiling at output weights 0.25, 1, 2, 4, 8 and 16 accepts all 72 candidates and leaves every variant's shape unchanged. Static recompilation on the records gives zero

median saving against all-branches cover execution (52 percent at most). Uniform model cells save 16 percent at the median and 59 percent at most.

Certified mode. The main-study median certified cost is 6.47 times original usage, or 2.70 times the heuristic shaped usage, and it varies by two orders of magnitude across workflows (Table 5). Flat workflows (routing, switch, email, orchestrator) cost two to four times their originals. Workflows that feed generated text into later prompts cost far more: chaining reaches about 104× and calculator about 47×. The cause is compounding worst cases. A response of visible cap k is bounded by $128k$ bytes, the longest native token piece, and under $\alpha = 1$ each such byte reserves an input token in every later request that contains the response. A typical response uses a few bytes per token, so most of the reserved envelope is filler. This is the price of an admission proof that holds for *every* guarded trajectory, with no lookahead. Three levers reduce it, each with a stated cost:

- a smaller response buffer, which clips more responses;
- a tokenizer constant below one token per byte, which byte-level BPE does not provide for arbitrary text;
- heuristic admission, which keeps the per-call refusal channel of Section 4.5.

The certified mode is thus practical for shallow workflows. For deep chains it quantifies what a worst-case guarantee costs.

Timing. Shaping does not equalise time. A retained latency-augmented attack reaches 100 percent balanced accuracy on one shaped workflow, which is why the observer excludes per-event timestamps.

6.5. RQ5: Toward the serving layer

Isolated padding makes the wrapper's report truthful but leaves two native submissions visible. We implemented a separate greedy prototype that issues one `llama_decode` batch of exactly 128 tokens. Useful request tokens occupy sequence 0 and filler occupies sequence 1, each with its own positions, and only the last useful token requests logits. Subsequent decode submissions contain one token each. Sequence membership isolates attention through the native batch interface [22]. The experiment covers two model families, three

Table 6: Native-prefill prototype, 12 cases per configuration. Single: one exact 128-token prefill. Equal: visible output equals natural unpadded generation. Metadata: sequence IDs recover the useful request length.

Native microbatch	Done	Single	Equal	Metadata
128	12	12	11	12
1	12	12	12	12
128 + cache removal	12	12	11	12

prompt bodies (empty, short, multilingual), visible caps 4 and 16, and forced length 24, on CPU. Table 6 keeps every attempt.

Natural native-size traces take four distinct values per model; the shaped projection takes one (128 followed by 23 single-token decodes). Report normalization above the API cannot produce this projection. Two negative results bound the claim. First, sequence and logit metadata still identify the useful tokens in every case, so a server that sees full batch metadata learns the request length. Second, with microbatch 128 one Qwen case changes its visible output, and removing the cover cache does not repair it. Microbatch 1 preserves all twelve outputs. TransparentOn is therefore a property of a serving configuration, not of attention isolation alone, and it must be established for each configuration.

6.6. Grader contracts: a categorical corrective-RAG ablation

The primary Qwen corrective-RAG grader mixes numeric-score instructions with a yes/no parser. Of its 600 grades, 484 are zeros, 34 ones, five *yes* and 77 *no*, so almost every retrieved document is discarded. A separate model contract pairs a categorical prompt with a grammar that admits only `{"score":"yes"}` or `{"score":"no"}`. Answer generation, retrieval and the rule "search if any grade is not *yes*" are unchanged. All 150 pairs preserve saved fields, including the four grades, with one shaped trace. All 600 grades are valid (62 *yes*, 538 *no*). Inherited accuracy rises from 57/150 to 84/150: 28 answers improve and one regresses (exact McNemar $p = 1.1 \times 10^{-7}$). Routing barely changes (149 of 150 questions still search), so the gain comes from keeping the documents now graded relevant (62 kept documents instead of 5). The ablation shows that a constrained grammar is a distinct model contract that shaping handles like any other. It does not separate the effects of the prompt and the grammar.

6.7. Threats to validity

Internal. Equal saved fields compare the variables each workflow declares as kept, including its decision variables and final output. Intermediate values are covered by trace equality and by the retained per-call request and response records. Greedy decoding makes runs deterministic per seed; sampling is covered by the theorems' per-tape statement but not exercised. Both studies use one machine, so GPU nondeterminism across devices is not sampled. The native prototype's counterexample shows that serving configuration can matter.

Construct. Balanced accuracy of one classifier family is a lower bound on leakage, not a measure of it. We therefore rest the security claim on the theorems and report classifier results only as motivation.

External. The workflows are authored bounded ports with small open models, and task accuracies are low. Shaping's preservation guarantee does not depend on accuracy. Its cost does depend on how much the branches of a workflow differ.

Reproducibility. Model weights and the native runtime are external dependencies pinned by hash. All other inputs, raw records and failed attempts are in the artifact.

7. Related work

Language-based information flow. Noninterference and its type-based enforcement are classical [23, 24, 25]. The program-counter security model formalises control-flow observations [26]. Agat's transformation, constant-resource typing and synthesis, and FaCT eliminate resource-dependent observations [8, 9, 12], and static analyses such as Blazer and Themis detect such channels [27, 28]. OrchLang applies balancing to operations whose costs are counted per model contract and whose inputs are generated text. Its contribution lies in the contracts and the admission certificate, not in a new type system.

Verified and validated compilation. Verified compilers prove every compilation correct [29], and constant-time preservation has been verified for a realistic C compiler [30, 31]. Translation validation instead checks each output [11, 13]. OrchLang validates each repair, and it additionally binds the validator's original and policy to source bytes, which closes a policy-omission attack that an unbound interface admits.

Resource reasoning and budgets. RelCost and probabilistic relational Hoare logics reason about relative and expected costs in general [15, 16]. Our budget theorem is a corollary of a certified exact shape, and the admission certificate is a specialised, decidable invariant for text-valued stores. Token-budget catalogues and workflow cost bounds address spending without confidentiality [17, 18, 32, 33].

Traffic shaping and timing. Supersequence defenses, Tamaraw and Pacer shape network traffic [10, 34, 14], and countermeasure studies show how partial shaping fails [35]. Predictive mitigation bounds timing leakage with delayed releases [36, 37]. Providers have mitigated token-length channels in streams by padding [38, 39]. These defenses target a network or clock observer. Ours targets the exported logical record, and Section 6.5 shows what remains below it.

LLM programs and agents. LMQL, DSPy and SGLang structure model calls as programs [40, 41, 42]; their constraints concern response languages and performance, not usage observations. CaMeL, FIDES and Hard-KLAIM enforce integrity, information flow or authority for agents [43, 44, 45], and SICC compiles policy-relative inter-agent messages [46]. These systems control which content flows where. OrchLang controls the usage metadata that remains when content is hidden, and the two properties compose.

8. Limitations

The theorems are about the core semantics under explicit serving contracts. The source reader, lowerer and native serving code are trusted; we check them by agreement, round trip and per-request monitoring, not by proof. The certified guarantee covers inputs within the declared byte guard. Refusing larger inputs is a public decision on input size, but it is still observable. The byte interface clips tool results identically in both arms, so the bounded original can differ from the unbounded application (Section 6.3 quantifies this). Worst-case byte envelopes compound through chains of generated text, which makes certified execution expensive for deep workflows (Section 6.4). Successful context admission does not establish device-memory availability, as the retained out-of-memory run at a 32,768-token context shows. Side effects, concurrency and stateful tools are outside the semantics.

The application corpus consists of authored bounded ports of public workflow examples run with small local models. Their task accuracies are

low, and shaping preserves them rather than improving them. Leakage is measured with one classifier family on these corpora; the formal theorems, not the classifiers, carry the security claim. The native prototype fixes a batch-size projection but not sequence metadata, and timing protection would require paced release with its own leakage and deadline analysis.

9. Conclusion

OrchLang turns a hidden-content LLM workflow into one whose exported usage record is independent of secrets. The guarantee is checked per repair against a source-derived policy. A static certificate ensures in advance that the shaped record is truthful and refusal-free. The proofs need only restricted serving contracts that real non-truncating padders satisfy, together with a tokenizer segmentation property that the runtime monitors. The certified evaluation shows that these guarantees hold on real models at a measurable cost, and the native prototype marks where a stronger serving refinement must begin.

Data and software availability

The artifact contains the Lean development, Python sources, workflow sources, datasets, all execution records (including failed and preliminary attempts), protocols, generated tables and SHA-256 integrity manifests, together with the complete Git history. Model weights are obtained separately using the recorded revisions and hashes. The development history is public at https://github.com/guyoverclocked/orchlang-compiler-design-project. The review archive will be deposited with a persistent identifier on acceptance.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used AI coding and writing assistants (Claude, Anthropic; Codex, OpenAI) to help implement experiment and proof scaffolding, run checks, and edit the language of the manuscript. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] OpenTelemetry, Semantic conventions for generative AI spans (status: Development), https://github.com/open-telemetry/semantic-conventions-genai, accessed 23 October 2026 (2026).

[2] LangChain, Prevent logging of sensitive data in traces (LangSmith documentation), https://docs.langchain.com/langsmith/mask-inputs-outputs, accessed 23 October 2026 (2026).

[3] Langfuse, Masking (langfuse documentation), https://langfuse.com/docs/observability/features/masking, accessed 23 October 2026 (2026).

[4] Helicone, Omit logs (helicone documentation), https://docs.helicone.ai/features/advanced-usage/omit-logs, accessed 23 October 2026 (2026).

[5] R. Weiss, D. Ayzenshteyn, G. Amit, Y. Mirsky, What was your prompt? a remote keylogging attack on AI assistants, in: 33rd USENIX Security Symposium (USENIX Security 24), 2024, pp. 3367–3384.
URL https://www.usenix.org/conference/usenixsecurity24/presentation/weiss

[6] T. Zhang, G. Saileshwar, D. Lie, Time will tell: Timing side channels via output token count in large language models, arXiv:2412.15431 (2024).

[7] T.-R. Chen, Sensitive-topic leakage through LLM routing metadata: Measurement and mitigation, arXiv:2610.09981 (2026).

[8] J. Agat, Transforming out timing leaks, in: Proceedings of the 27th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL), 2000, pp. 40–53. doi:10.1145/325694.325702.

[9] V. C. Ngo, M. Dehesa-Azuara, M. Fredrikson, J. Hoffmann, Verifying and synthesizing constant-resource implementations with types, in: IEEE Symposium on Security and Privacy (S&P), 2017, pp. 710–728. doi:10.1109/SP.2017.53.

[10] T. Wang, X. Cai, R. Nithyanand, R. Johnson, I. Goldberg, Effective attacks and provable defenses for website fingerprinting, in: 23rd USENIX Security Symposium, 2014, pp. 143–157.

[11] A. Pnueli, M. Siegel, E. Singerman, Translation validation, in: Tools and Algorithms for the Construction and Analysis of Systems (TACAS), Vol. 1384 of LNCS, 1998, pp. 151–166. doi:10.1007/BFb0054170.

[12] S. Cauligi, G. Soeller, B. Johannesmeyer, F. Brown, R. S. Wahby, J. Renner, B. Grégoire, G. Barthe, R. Jhala, D. Stefan, FaCT: A DSL for timing-sensitive computation, in: Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2019, pp. 174–189. doi:10.1145/3314221.3314605.

[13] G. C. Necula, Translation validation for an optimizing compiler, in: Proceedings of the ACM SIGPLAN 2000 Conference on Programming Language Design and Implementation (PLDI), 2000, pp. 83–94. doi:10.1145/349299.349314.

[14] A. Mehta, M. Alzayat, R. D. Viti, B. B. Brandenburg, P. Druschel, D. Garg, Pacer: Comprehensive network side-channel mitigation in the cloud, in: 31st USENIX Security Symposium, 2022, pp. 2819–2838.
URL https://www.usenix.org/conference/usenixsecurity22/presentation/mehta

[15] E. Çiçek, G. Barthe, M. Gaboardi, D. Garg, J. Hoffmann, Relational cost analysis, in: Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL), 2017, pp. 316–329. doi:10.1145/3009837.3009858.

[16] M. Avanzini, G. Barthe, D. Davoli, B. Grégoire, A quantitative probabilistic relational hoare logic, Proceedings of the ACM on Programming Languages 9 (POPL) (2025) 1167–1195. doi:10.1145/3704876.

[17] S. Khan, Token budgets: An empirical catalog of 63 LLM-agent budget-overrun incidents, with an affine-typed Rust mitigation as a case study, arXiv:2606.04056 (2026).

[18] K. Dickerson, Statically bounding resource accumulation in LLM agent workflows: The cost case, Author-hosted report, https://loom.technology/research/bounded-llm-cost/, 18 May 2026; not peer reviewed (2026).

[19] L. de Moura, S. Ullrich, The Lean 4 theorem prover and programming language, in: Automated Deduction – CADE 28, Vol. 12699 of LNCS, 2021, pp. 625–635. doi:10.1007/978-3-030-79876-5_37.

[20] Meta, Llama 3.2 1b and 3b instruct model cards, https://huggingface.co/meta-llama (2024).

[21] Qwen Team, Qwen2.5 technical report, arXiv:2412.15115 (2024).

[22] ggml-org, llama.cpp native batch interface, revision `9c2e0e491`, https://github.com/ggml-org/llama.cpp/blob/9c2e0e491/include/llama.h, accessed 29 October 2026 (2026).

[23] J. A. Goguen, J. Meseguer, Security policies and security models, in: IEEE Symposium on Security and Privacy, 1982, pp. 11–20. doi:10.1109/SP.1982.10014.

[24] D. Volpano, C. Irvine, G. Smith, A sound type system for secure flow analysis, Journal of Computer Security 4 (2–3) (1996) 167–187. doi:10.3233/JCS-1996-42-304.

[25] A. Sabelfeld, A. C. Myers, Language-based information-flow security, IEEE Journal on Selected Areas in Communications 21 (1) (2003) 5–19. doi:10.1109/JSAC.2002.806121.

[26] D. Molnar, M. Piotrowski, D. Schultz, D. Wagner, The program counter security model: Automatic detection and removal of control-flow side channel attacks, in: Information Security and Cryptology (ICISC 2005), Vol. 3935 of LNCS, 2006, pp. 156–168. doi:10.1007/11734727_14.

[27] T. Antonopoulos, P. Gazzillo, M. Hicks, E. Koskinen, T. Terauchi, S. Wei, Decomposition instead of self-composition for proving the absence of timing channels, in: Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2017, pp. 362–375. doi:10.1145/3062341.3062378.

[28] J. Chen, Y. Feng, I. Dillig, Precise detection of side-channel vulnerabilities using quantitative Cartesian Hoare logic, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS), 2017, pp. 875–890. doi:10.1145/3133956.3134058.

[29] X. Leroy, Formal verification of a realistic compiler, Communications of the ACM 52 (7) (2009) 107–115. doi:10.1145/1538788.1538814.

[30] G. Barthe, S. Blazy, B. Grégoire, R. Hutin, V. Laporte, D. Pichardie, A. Trieu, Formal verification of a constant-time preserving C compiler, Proceedings of the ACM on Programming Languages 4 (POPL) (2020). doi:10.1145/3371075.

[31] G. Barthe, S. Blazy, R. Hutin, D. Pichardie, Secure compilation of constant-resource programs, in: IEEE 34th Computer Security Foundations Symposium (CSF), 2021, pp. 1–12. doi:10.1109/CSF51468.2021.00020.

[32] Costwright contributors, Costwright source repository, https://github.com/eleata/costwright, source repository, not peer reviewed; accessed 24 October 2026 (2026).

[33] Typed Resources contributors, Typed resources source repository, https://github.com/eleata/typed-resources, source repository, not peer reviewed; accessed 24 October 2026 (2026).

[34] X. Cai, R. Nithyanand, T. Wang, R. Johnson, I. Goldberg, A systematic approach to developing and evaluating website fingerprinting defenses, in: Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS), 2014, pp. 227–238. doi:10.1145/2660267.2660362.

[35] K. P. Dyer, S. E. Coull, T. Ristenpart, T. Shrimpton, Peek-a-boo, I still see you: Why efficient traffic analysis countermeasures fail, in: IEEE Symposium on Security and Privacy (S&P), 2012, pp. 332–346. doi:10.1109/SP.2012.28.

[36] A. Askarov, D. Zhang, A. C. Myers, Predictive black-box mitigation of timing channels, in: Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS), 2010, pp. 297–307. doi:10.1145/1866307.1866341.

[37] D. Zhang, A. Askarov, A. C. Myers, Language-based control and mitigation of timing channels, in: Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2012, pp. 99–110. doi:10.1145/2254064.2254078.

[38] C. Martinho, M. Chen, Mitigating a token-length side-channel attack in our AI products, Cloudflare blog, https://blog.cloudflare.com/ai-side-channel-attack-mitigated (2024).

[39] OpenAI, API reference: `stream_options.include_obfuscation`, https://developers.openai.com/api/docs/api-reference/responses/get, accessed 23 October 2026 (2026).

[40] L. Beurer-Kellner, M. Fischer, M. Vechev, Prompting is programming: A query language for large language models, Proceedings of the ACM on Programming Languages 7 (PLDI) (2023) 1946–1969. doi:10.1145/3591300.

[41] O. Khattab, et al., DSPy: Compiling declarative language model calls into state-of-the-art pipelines, International Conference on Learning Representations (ICLR) (2024).

[42] L. Zheng, et al., SGLang: Efficient execution of structured language model programs, Advances in Neural Information Processing Systems (NeurIPS) (2024).

[43] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, F. Tramèr, Defeating prompt injections by design, arXiv:2503.18813 (2025).

[44] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, S. Zanella-Béguelin, Securing AI agents with information-flow control, arXiv:2505.23643 (2025).

[45] M. Aldinucci, A. Bracciali, I. Colonnelli, D. Medić, A. Mulone, L. Roversi, M. E. Santimaria, Hard-KLAIM: Hardening MCP-enabled agentic workflows with a locality-aware type-and-effect system for trace safety, Journal of Logical and Algebraic Methods in Programming (2026) 101191. In press, journal pre-proof; available online 9 October 2026. doi:10.1016/j.jlamp.2026.101191.

[46] J. Xu, L. Fan, Z. Wang, X. Li, H. Liu, Selection-invariant communication compilers for privacy-aware multi-agent LLM workflows, arXiv:2609.26076 (2026).

Appendix A. Earlier cohorts and the historical implementation

Earlier evidence remains immutable supplementary material and is never pooled with the present studies. An earlier main cohort retained 962 pairs; a later completion added 210 main and 512 Qwen pairs under a different implementation. Template and Unicode corrections were checked separately: a 48-pair workflow study, a 192-case primitive matrix and a 24-pair release check. A targeted comparison differed from older saved fields in 6 of 18 examples. A 24-pair byte-bounded pilot preceded the certified study of Section 6.3. The archive also retains `legacy/v1`, an earlier C++/Lean certificate analyzer with a different AST and policy. Its 106 C++ tests, 82 Python tests and four Lean files are excluded from the counts in this paper.

Appendix B. Reproduction map

Start with `publication/README.md`. With `LEAN` pointing at Lean 4.19.0 and `ORCHLANG_MODELS` at the pinned model configuration:

```
python publication/check_proofs.py --output out/proofs.json
python -m unittest discover -s tests
python experiments/source_binding_case.py --output out/policy.json
python experiments/serializer_roundtrip.py --output out/roundtrip.json
python experiments/correspondence.py --programs 150 --output out/corr
python experiments/differential.py --programs 1000 --runs 4 \
  --output out/differential.json
python experiments/homogeneous_study.py campaign --models $ORCHLANG_MODELS
    \
  --output out/certified --context 21504 --byte-bounded --response-bytes
     128
python publication/analyze_round3.py --certified out/certified \
  --output out/analysis
```

The heuristic study is reproduced by the same campaign driver without the byte-bounded flags at a 4,096-token context. The campaign driver requires a clean Git checkout and refuses to overwrite evidence. Manifests record every identity needed to compare a new run with ours.